AERO FLOW

Multi-Airport Data Engineering Platform

GRADUATION PROJECT DOCUMENTATION

Digital Egypt Pioneers Initiative (DEPI)

Phase 5 | Microsoft Data Engineer Track

Team Size: 6 Members

Project Information

Project Name: Aero Flow

Project Domain: Aviation Data Engineering

Project Type: End-to-End Data Engineering Platform

GitHub Repository: https://github.com/MoahmaedHazemSalem/Aero-Flow

---

Team Leader Information

Full Name: محمد حازم فهمي سالم

Student ID: 21003623

Role: Team Leader & Analytics Engineering

Project Team Members

No.	Full Name	Student ID
1	محمد حازم فهمي سالم (Team Leader)	21003623
2	عمر احمد مصطفى عبدالحليم	21157840
3	محمد مرتضي ابراهيم يوسف	21158325
4	محمد نادر محمدي صالح	21015391
5	شذى احمد معوض عبدالعواض	21172342
6	رؤى طاهر أبوبكر محمد	21176035

1. Project Planning & Management

1.1 Project Overview

Aero Flow is an end-to-end Data Engineering platform designed to collect, process, integrate, store, and analyze aviation data from multiple sources using a hybrid batch and streaming architecture.

The platform combines publicly available aviation datasets, historical flight records, aviation weather observations, and simulated flight operations into a unified data processing environment.

Aero Flow aims to demonstrate how modern Data Engineering technologies can be used to build reliable data pipelines capable of processing both historical information and continuously generated operational events.

The platform will use Python and external APIs for collecting aviation data, FastAPI and PostgreSQL for simulating flight operations, Apache Kafka for event streaming, and Apache Spark Structured Streaming for processing incoming events.

HDFS, Apache Parquet, and Apache Iceberg will be used to organize and manage data in a structured Data Lake environment.

SQL, Spark SQL, and dbt will support analytical transformations, historical tracking, data modeling, and data quality validation.

An interactive dashboard developed using Streamlit and Plotly will provide visualizations of airport information, flight activities, operational performance, weather observations, and pipeline monitoring results.

The system will distinguish between real historical aviation datasets and simulated flight operations to maintain data transparency and avoid presenting generated events as actual airline activities.

The proposed solution focuses on building a practical and scalable Data Engineering prototype rather than a commercial airline management system.

Project Vision:

To develop a unified aviation Data Engineering platform that demonstrates reliable data ingestion, real-time processing, distributed storage, historical data management, and analytical reporting across multiple airports.

Target Audience:

- Data Engineers
- Analytics Engineers
- Data Analysts
- Aviation Data Researchers
- Data Teams working with operational and historical datasets
- Students and developers interested in modern Data Engineering technologies

1.2 Problem Statement

Aviation data is generated across multiple independent systems and services, including airport reference databases, flight performance datasets, operational systems, and aviation weather services.

These data sources frequently differ in their formats, schemas, update frequencies, and identifier conventions, making it difficult to integrate them into a unified analytical environment.

Historical flight datasets may contain missing values, inconsistent records, and different airport identifiers. Weather observations may also require careful alignment with flight locations and timestamps.

Additionally, flight operations are continuously changing due to schedule modifications, delays, cancellations, and gate updates.

Traditional batch processing alone may not provide a suitable solution for handling continuously generated operational events.

Combining batch and streaming data introduces further challenges related to event duplication, delayed processing, data consistency, storage management, historical tracking, and recovery from processing failures.

Another important challenge is preserving historical changes in flight information. When existing records are overwritten, previous operational states may be lost, reducing the ability to analyze how flight conditions changed over time.

Data Engineering teams therefore need an organized architecture capable of collecting heterogeneous datasets, processing operational events, maintaining historical information, validating data quality, and providing reliable analytical outputs.

Aero Flow aims to address these challenges by implementing an integrated Data Engineering platform that combines batch ingestion, event streaming, distributed processing, structured storage, analytical transformations, and interactive data visualization.

The platform will demonstrate the complete data lifecycle using publicly available aviation datasets and simulated operational events.

1.3 Project Objectives

The main objective of Aero Flow is to develop a reliable end-to-end aviation Data Engineering platform that integrates batch and streaming data processing into a unified analytical environment.

The specific objectives of the project are:

1. **Multi-Source Data Integration:** Collect and integrate aviation information from publicly available airport datasets, historical flight records, and aviation weather APIs.
2. **Automated Batch Ingestion:** Develop Python-based pipelines to extract, validate, clean, and standardize aviation datasets.
3. **Flight Operations Simulation:** Implement a flight simulation system using Python, FastAPI, and PostgreSQL to generate flight activities and operational changes.
4. **Real-Time Event Streaming:** Use Apache Kafka to publish and distribute simulated flight events, including delays, cancellations, and gate modifications.
5. **Stream Processing:** Implement Spark Structured Streaming to process incoming flight events, validate records, handle duplicates, and generate streaming results.
6. **Distributed Data Storage:** Build a structured Data Lake using HDFS, Apache Parquet, and Apache Iceberg to manage raw, processed, and curated aviation datasets.
7. **Analytical Data Modeling:** Develop SQL and dbt transformations, fact tables, dimension tables, and analytical data marts.
8. **Historical Change Tracking:** Apply Slowly Changing Dimension Type 2 (SCD Type 2) techniques to preserve relevant historical changes in flight information.

9. **Data Quality and Reliability:** Implement automated validation checks, error handling, recovery mechanisms, and consistency testing.
10. **Interactive Monitoring and Performance Evaluation:** Develop a Streamlit dashboard and evaluate pipeline performance using latency, throughput, data completeness, and processing reliability metrics.

1.4 Project Scope

The scope of Aero Flow is to design and implement an aviation Data Engineering prototype capable of collecting, processing, storing, and analyzing information related to multiple airports.

The platform is not restricted to specific airports or a fixed geographical region.

The project will integrate real aviation datasets with simulated operational events to demonstrate a complete batch and streaming architecture.

In Scope

The project will include the following functionalities:

1. **Aviation Data Collection:** Collect airport reference information, historical flight data, and aviation weather observations.
2. **Batch Data Ingestion:** Develop Python pipelines for extracting, cleaning, and validating aviation datasets.
3. **Flight Operations Simulation:** Generate simulated flight records, schedule modifications, delays, cancellations, and gate changes.
4. **Operational Data Management:** Store and manage simulated flight information using PostgreSQL and FastAPI.
5. **Event Streaming:** Publish simulated flight events through Apache Kafka.
6. **Stream Processing:** Process operational events using Apache Spark Structured Streaming.
7. **Data Lake Storage:** Organize raw, processed, and curated data using HDFS, Parquet, and Iceberg.
8. **Data Integration:** Combine historical flight records, airport reference information, weather observations, and simulated operational events.
9. **Data Warehousing and Analytics:** Develop analytical models using SQL, Spark SQL, and dbt.

10. **Historical Data Management:** Preserve relevant changes in flight-related information using SCD Type 2.
11. **Data Quality Validation:** Apply automated checks for completeness, consistency, duplicate detection, and referential integrity.
12. **Interactive Dashboard:** Display airport information, flight events, operational indicators, weather information, and analytical results using Streamlit and Plotly.
13. **Pipeline Monitoring:** Present available processing status, quality indicators, and execution results.
14. **Testing and Evaluation:** Evaluate pipeline functionality, reliability, recovery, latency, throughput, and data completeness.

Out of Scope

The following functionalities are outside the initial project scope:

1. Direct integration with real airport air traffic control systems.
2. Real-world control of aircraft movements or airport operations.
3. Airline ticket booking and passenger reservation management.
4. Development of a commercial airline management application.
5. Guaranteeing continuous availability of all external aviation data sources.
6. Treating simulated flight events as verified real-world airline operations.
7. Production deployment for safety-critical aviation environments.
8. Providing complete global real-time flight tracking without appropriate external data sources.
9. Advanced machine learning-based flight delay prediction.
10. Implementing a complete enterprise-scale aviation management platform.

The final implementation will prioritize a functional and testable Data Engineering prototype that demonstrates the core capabilities of the proposed architecture.

1.5 Project Deliverables

The Aero Flow project is expected to produce the following deliverables:

1. **Functional Data Engineering Platform:** A working prototype integrating aviation data ingestion, operational simulation, streaming processing, data storage, analytical modeling, and visualization.
2. **Aviation Data Ingestion Pipelines:** Python scripts for collecting and preparing airport reference data, historical flight information, and aviation weather observations.
3. **Flight Operations Simulation System:** A working FastAPI and PostgreSQL application capable of generating and managing simulated flight events.
4. **Event Streaming Pipeline:** An Apache Kafka-based pipeline for publishing and receiving simulated operational events.
5. **Stream Processing Component:** Spark Structured Streaming jobs for processing, validating, and transforming incoming flight events.
6. **Data Lake and Lakehouse Storage:** An organized storage environment using HDFS, Parquet, and Iceberg.
7. **Data Integration Pipelines:** Apache Spark jobs for combining relevant data from different aviation sources.
8. **Analytical Data Models:** SQL and dbt models containing staging tables, fact tables, dimension tables, and analytical data marts.
9. **Historical Tracking Component:** SCD Type 2 implementation for preserving relevant historical flight changes.
10. **Data Quality Validation:** Automated checks and documented results for data completeness, consistency, and integrity.
11. **Interactive Dashboard:** A Streamlit dashboard presenting aviation information, operational performance, historical changes, and monitoring results.
12. **Testing and Evaluation Results:** Documented testing scenarios covering functionality, reliability, recovery, and processing performance.
13. **Source Code Repository:** A GitHub repository containing the project source code, configuration examples, tests, setup instructions, and documentation.
14. **Technical Documentation:** Documentation describing the system architecture, components, data sources, implementation approach, and testing methodology.

15. **Final Project Presentation:** A presentation and demonstration explaining the project problem, proposed solution, architecture, implementation, testing, and results.

1.6 Project Methodology

The Aero Flow project will follow an incremental development methodology based on clear technical responsibilities and sequential component handovers.

The project will be divided into manageable development stages, with each stage producing a functional and testable component.

Every team member will participate directly in implementing and testing an assigned technical component.

The project will follow the phases described below.

1. Requirements Analysis:

Identify the main aviation data integration challenges, define the functional and non-functional requirements, and determine the expected capabilities of the system.

2. System Design:

Define the system architecture, data models, processing pipelines, component interfaces, and communication between the different Data Engineering technologies.

3. Aviation Data Collection and Simulation:

Develop batch ingestion pipelines for public aviation datasets and implement a simulated flight operations environment.

4. Streaming and Data Processing:

Configure Apache Kafka, implement flight event publishing, and develop Spark Structured Streaming pipelines.

5. Data Storage and Analytical Modeling:

Implement the Data Lake environment, integrate aviation datasets, and develop analytical transformations and historical tracking models.

6. Dashboard and System Integration:

Develop the Streamlit dashboard and connect the analytical outputs to the monitoring interface.

7. Testing and Evaluation:

Perform component testing, end-to-end integration testing, data quality validation, recovery testing, and performance evaluation.

8. Documentation and Presentation:

Document the implemented components, testing results, architecture, and final demonstration.

The team will use GitHub for version control and collaboration.

Each component will be tested before being handed over to the next responsible member.

After all components are integrated, the entire team will participate in final system testing and preparation for the graduation project presentation.

1.5 Project Deliverables

The Aero Flow project is expected to produce the following deliverables:

1. **Functional Data Engineering Platform:** A working prototype integrating aviation data ingestion, operational simulation, streaming processing, data storage, analytical modeling, and visualization.
2. **Aviation Data Ingestion Pipelines:** Python scripts for collecting and preparing airport reference data, historical flight information, and aviation weather observations.
3. **Flight Operations Simulation System:** A working FastAPI and PostgreSQL application capable of generating and managing simulated flight events.
4. **Event Streaming Pipeline:** An Apache Kafka-based pipeline for publishing and receiving simulated operational events.
5. **Stream Processing Component:** Spark Structured Streaming jobs for processing, validating, and transforming incoming flight events.
6. **Data Lake and Lakehouse Storage:** An organized storage environment using HDFS, Parquet, and Iceberg.
7. **Data Integration Pipelines:** Apache Spark jobs for combining relevant data from different aviation sources.
8. **Analytical Data Models:** SQL and dbt models containing staging tables, fact tables, dimension tables, and analytical data marts.
9. **Historical Tracking Component:** SCD Type 2 implementation for preserving relevant historical flight changes.

10. **Data Quality Validation:** Automated checks and documented results for data completeness, consistency, and integrity.
11. **Interactive Dashboard:** A Streamlit dashboard presenting aviation information, operational performance, historical changes, and monitoring results.
12. **Testing and Evaluation Results:** Documented testing scenarios covering functionality, reliability, recovery, and processing performance.
13. **Source Code Repository:** A GitHub repository containing the project source code, configuration examples, tests, setup instructions, and documentation.
14. **Technical Documentation:** Documentation describing the system architecture, components, data sources, implementation approach, and testing methodology.
15. **Final Project Presentation:** A presentation and demonstration explaining the project problem, proposed solution, architecture, implementation, testing, and results.

1.6 Project Methodology

The Aero Flow project will follow an incremental development methodology based on clear technical responsibilities and sequential component handovers.

The project will be divided into manageable development stages, with each stage producing a functional and testable component.

Every team member will participate directly in implementing and testing an assigned technical component.

The project will follow the phases described below.

1. Requirements Analysis:

Identify the main aviation data integration challenges, define the functional and non-functional requirements, and determine the expected capabilities of the system.

2. System Design:

Define the system architecture, data models, processing pipelines, component interfaces, and communication between the different Data Engineering technologies.

3. Aviation Data Collection and Simulation:

Develop batch ingestion pipelines for public aviation datasets and implement a simulated flight operations environment.

4. Streaming and Data Processing:

Configure Apache Kafka, implement flight event publishing, and develop Spark Structured Streaming pipelines.

5. Data Storage and Analytical Modeling:

Implement the Data Lake environment, integrate aviation datasets, and develop analytical transformations and historical tracking models.

6. Dashboard and System Integration:

Develop the Streamlit dashboard and connect the analytical outputs to the monitoring interface.

7. Testing and Evaluation:

Perform component testing, end-to-end integration testing, data quality validation, recovery testing, and performance evaluation.

8. Documentation and Presentation:

Document the implemented components, testing results, architecture, and final demonstration.

The team will use GitHub for version control and collaboration.

Each component will be tested before being handed over to the next responsible member.

After all components are integrated, the entire team will participate in final system testing and preparation for the graduation project presentation.

1.7 Team Roles & Responsibilities

The Aero Flow project will be developed by six team members, with each member responsible for implementing, testing, and documenting a specific technical component.

The development process will follow a sequential implementation and handover approach.

All team members will contribute through practical coding, testing, and integration.

1. Real Aviation Data & Batch Ingestion

Assigned Member: رؤى طاهر أبوبكر محمد

Tools: Python, APIs

Responsibilities:

1. Collect airport reference data from OurAirports.
2. Obtain historical flight records from BTS TranStats.
3. Integrate aviation weather observations using the Aviation Weather API.
4. Develop Python-based batch ingestion pipelines.
5. Perform data cleaning, validation, and standardization.
6. Prepare consistent data schemas and airport reference datasets.
7. Handle missing values, duplicate records, and ingestion errors.
8. Test and document the data collection pipelines.

Expected Deliverable:

A functional batch ingestion system that provides validated aviation datasets, documented schemas, and reusable Python scripts.

Handover To: محمد نادر محمدي صالح.

2. Flight Operations & Simulation

Assigned Member: محمد نادر محمدي صالح

Tools: Python, FastAPI, PostgreSQL

Responsibilities:

1. Design and implement the operational PostgreSQL database.
2. Develop a Python-based Flight Simulator.
3. Create FastAPI endpoints for managing flight operations.
4. Support flight creation, schedule updates, delays, cancellations, and gate changes.
5. Integrate airport reference data provided by the data ingestion component.
6. Generate structured operational flight events.
7. Implement input validation and error handling.
8. Test the database operations, API endpoints, and simulation functionality.

Expected Deliverable:

A working flight operations simulation system capable of generating, updating, storing, and documenting simulated flight events.

Handover To: شذى احمد معوض عبدالعواض.

3. Event Streaming

Assigned Member: شذى احمد معوض عبدالعواض

Tools: Apache Kafka, Python, Spark Structured Streaming

Responsibilities:

1. Configure Apache Kafka and the required event topics.
2. Develop Python producers to publish simulated flight events.
3. Implement Spark Structured Streaming pipelines to consume Kafka events.
4. Validate, clean, and transform incoming event data.
5. Handle duplicate records and late-arriving events.
6. Implement streaming aggregations and event-time processing.
7. Configure checkpointing and recovery mechanisms.
8. Evaluate streaming latency, throughput, and processing reliability.

Expected Deliverable:

A functional streaming pipeline that transfers simulated flight events through Kafka and processes them using Spark Structured Streaming.

Handover To: محمد مرتضي ابراهيم يوسف.

4. Data Integration & Lakehouse Engineering

Assigned Member: محمد مرتضي ابراهيم يوسف

Tools: Apache Spark, Spark SQL, HDFS, Parquet, Apache Iceberg

Responsibilities:

1. Configure distributed data storage using HDFS.
2. Design and implement Raw, Processed, and Curated data layers.
3. Store aviation datasets and streaming events using Apache Parquet.
4. Develop Spark processing jobs for data transformation and integration.

5. Integrate historical flight records, airport reference data, weather observations, and simulated events.
6. Implement curated analytical tables using Apache Iceberg.
7. Perform data reconciliation, validation, and consistency checks.
8. Test data storage, recovery, reprocessing, and integration performance.

Expected Deliverable:

An integrated Data Lake and Lakehouse environment containing validated historical and streaming aviation datasets organized for analytical processing.

Handover To: محمد حازم فهمي سالم.

5. Analytics & Historical Modeling

Assigned Member: محمد حازم فهمي سالم (Team Leader)

Tools: SQL, Spark SQL, dbt

Responsibilities:

1. Design analytical data models for the Aero Flow platform.
2. Develop staging models for integrated aviation datasets.
3. Build fact and dimension tables for flight and airport analytics.
4. Implement SQL-based transformations using dbt.
5. Apply Slowly Changing Dimension Type 2 (SCD Type 2) to preserve relevant historical data changes.
6. Calculate aviation performance indicators, including flight delays, cancellations, and operational trends.
7. Implement automated data quality and referential integrity tests.
8. Prepare analytical data marts for visualization.
9. Validate analytical results and document the implemented transformations.

Expected Deliverable:

A structured analytical data layer containing SQL and dbt transformations, fact and dimension tables, historical tracking, data quality tests, and aviation performance indicators.

Handover To: عمر احمد مصطفى عبدالحليم.

6. Dashboard & System Visibility

Assigned Member: عمر احمد مصطفى عبدالحليم

Tools: Streamlit, Plotly, Python

Responsibilities:

1. Develop the interactive Aero Flow monitoring dashboard.
2. Visualize airport networks and available flight routes.
3. Implement a flight information board displaying flight schedules and statuses.
4. Present flight performance indicators and analytical results.
5. Display aviation weather observations.
6. Develop views for simulated gate assignments and flight history.
7. Visualize pipeline monitoring metrics and data quality results.
8. Connect dashboard components to processed analytical datasets.
9. Test dashboard functionality, data accuracy, and user interaction.

Expected Deliverable:

A functional interactive dashboard presenting aviation data, simulated flight events, analytical insights, historical changes, and pipeline monitoring information.

Final Handover: Complete system to the entire project team for integration testing and presentation.

Implementation & Handover Process

The project will follow the implementation sequence:

رؤى طاهر أبوبكر محمد → محمد نادر محمدي صالح → شذى احمد معوض عبدالعواض → محمد مرتضي ابراهيم يوسف → محمد حازم فهمي سالم → عمر احمد مصطفى عبدالحليم

Every handover will include:

1. Functional source code uploaded to the shared GitHub repository.
2. Required configuration files and setup instructions.
3. Documented inputs, outputs, and data schemas.

4. Successful component tests and validation results.
5. Sample data or events for integration testing.
6. Confirmation that the receiving member can run the delivered component.

After the six components have been developed, the entire team will participate in end-to-end testing and issue resolution.

The Team Leader will also coordinate project progress while maintaining direct responsibility for implementing the Analytics & Historical Modeling component.

1.8 Project Timeline

Aero Flow will follow a structured development plan based on sequential component implementation, testing, and integration.

The preliminary planning period extends from late October 2026 to the final scheduled training session on December 8, 2026.

This period is used for initial project planning and does not represent a confirmed final submission deadline.

Phase	**Main Activities**	**Proposed Schedule**
1. Aviation Data Collection	Collect reference, historical, and weather data. Develop and validate batch ingestion pipelines.	Week 1
2. Flight Operations & Simulation	Develop PostgreSQL database, FastAPI endpoints, and Python Flight Simulator.	Week 2
3. Event Streaming & Processing	Configure Kafka, develop event producers, and implement Spark Structured Streaming.	Week 3
4. Data Lake & Integration	Implement HDFS, Parquet, Iceberg, and Spark-based data integration.	Week 4
5. Analytics & Historical Modeling	Develop SQL and dbt models, historical tracking, and quality tests.	Week 5

6. Dashboard Development	Develop the Streamlit dashboard and connect analytical outputs.	Week 6
7. Final Integration & Evaluation	Conduct end-to-end testing, resolve issues, and prepare demonstration materials.	Final available period

The proposed schedule is subject to team review and adjustment based on technical requirements, available computing resources, and component completion status.

Each team member will be responsible for completing and testing the assigned component before the next stage is integrated.

The final timeline will be confirmed by the team.

1.9 Risk Management & Mitigation

The Aero Flow project may face technical and organizational risks related to data availability, processing reliability, integration complexity, and project coordination.

The following potential risks and mitigation strategies have been identified.

Potential Risk	**Potential Impact**	**Mitigation Strategy**
External Data Source Unavailability	Interruptions in aviation data collection.	Maintain downloaded datasets, apply error handling, and retry failed requests when appropriate.
Inconsistent Data Formats	Difficulties integrating data from multiple sources.	Standardize schemas, validate data types, and maintain airport identifier mappings.
Missing or Invalid Records	Incorrect analytical results.	Implement automated validation and documented cleaning procedures.
Streaming Pipeline Failures	Interruptions in event processing.	Implement checkpointing, error handling, and controlled recovery tests.
Duplicate or Late Events	Incorrect event counts and flight histories.	Use event identifiers, duplicate handling, and event-time processing.
Data Storage Issues	Delays in processing or integration.	Test HDFS, Parquet, Iceberg, and Spark configurations before full integration.

Historical Tracking Errors	Incorrect representation of previous flight states.	Validate SCD Type 2 logic and historical records using controlled test cases.
Limited Computing Resources	Reduced system performance.	Optimize processing configurations and use manageable test datasets.
Integration Delays	Delays in subsequent development stages.	Define interfaces clearly, test handovers, and resolve issues early.
Project Schedule Delays	Incomplete project deliverables.	Monitor progress, prioritize essential requirements, and coordinate team activities.

Risk management will be reviewed throughout project development.

Each team member will be responsible for identifying and resolving technical issues related to their assigned component.

1.10 Project Success Metrics (KPIs)

The success of Aero Flow will be evaluated based on the functionality, reliability, data quality, and performance of the implemented Data Engineering platform.

KPI	**Measurement Method**
Data Ingestion Success	Compare successful ingestion operations with total attempted operations.
Data Processing Latency	Measure the time required for flight events to pass through the processing pipeline.
Data Throughput	Measure the number of events processed per second.
Data Completeness	Compare expected source records with successfully processed records.
Data Quality	Evaluate automated validation results for missing, invalid, inconsistent, and duplicate records.
Historical Data Accuracy	Verify that relevant previous flight states are correctly preserved using SCD Type 2.
Duplicate Handling	Check that repeated events do not produce incorrect final outputs.

Fault Recovery	Test whether processing can resume correctly after controlled failures.
Data Integration Accuracy	Verify that relevant datasets are correctly linked using valid identifiers and timestamps.
Dashboard Accuracy	Compare displayed information with underlying analytical results.

The project will be considered successful when its main components function together, the required features are demonstrated, and the integrated pipelines pass the agreed validation and testing procedures.

Actual performance values will be measured during implementation and testing.

Target Values: To Be Confirmed.

2. Literature Review

2.1 Existing Solutions and Related Work

Several existing aviation data platforms and publicly available services provide useful information for collecting and analyzing aviation datasets.

These solutions were reviewed to identify their capabilities, strengths, limitations, and relevance to Aero Flow.

The reviewed sources include OpenSky Network, BTS TranStats, OurAirports, and Aviation Weather Center.

1. OpenSky Network

OpenSky Network provides aircraft tracking information and historical air traffic data through research-oriented services and APIs.

Advantages:

- Provides access to aircraft position and air traffic information.
- Supports research involving historical aviation data.
- Offers programmatic access through documented APIs.
- Supports the study of aircraft movements and air traffic patterns.

Limitations:

- Does not directly provide complete commercial flight schedules and delay information.
- API usage and historical data access are subject to limitations.
- Does not provide all the operational information required for the Aero Flow simulation.

2. BTS TranStats

The Bureau of Transportation Statistics provides historical airline performance datasets containing information about flight schedules, departure and arrival times, delays, cancellations, and other operational measurements.

Advantages:

- Provides detailed historical flight performance information.
- Includes useful measurements for delay and cancellation analysis.
- Supports analysis across airports, airlines, and time periods.
- Provides datasets suitable for Data Engineering projects.

Limitations:

- The selected on-time performance datasets primarily cover domestic flights within the United States.
- Data is published periodically rather than as a continuous live operational stream.
- Requires cleaning, transformation, and integration with other sources.

3. OurAirports

OurAirports provides publicly available airport reference datasets containing airport identifiers, names, geographical coordinates, countries, regions, and runway information.

Advantages:

- Provides structured airport reference information.

- Covers airports across different countries and regions.
- Offers downloadable CSV datasets.
- Supports integration through airport identifiers.

Limitations:

- Focuses on airport reference information rather than operational flight events.
- Does not provide comprehensive live flight delay or cancellation information.
- Some records may contain missing identifiers or incomplete attributes.

4. Aviation Weather Center

The Aviation Weather Center provides aviation weather observations and related information through documented services and APIs.

Advantages:

- Provides aviation weather observations and related data.
- Supports programmatic access through APIs.
- Offers machine-readable data formats such as JSON.
- Supports weather-related analysis when relevant location and time information are available.

Limitations:

- Focuses on meteorological data rather than flight operations.
- API requests are subject to usage restrictions.
- Weather information requires careful matching with airport identifiers and timestamps.
- Historical weather coverage may not match every flight record.

2.2 Comparison with Aero Flow

The reviewed solutions provide valuable aviation data, but each primarily focuses on a specific category of information or service.

Aero Flow differs by focusing on the integration, processing, storage, and analytical preparation of information from multiple aviation data sources.

The proposed platform will combine historical and reference datasets with simulated operational flight events.

Unlike individual data providers, Aero Flow will demonstrate how batch ingestion and streaming processing can operate within a unified Data Engineering architecture.

Apache Kafka and Spark Structured Streaming will process simulated flight events, while HDFS, Parquet, and Iceberg will support data storage and integration.

SQL and dbt will provide analytical transformations, historical tracking, and data quality validation.

Streamlit will provide an interactive interface for exploring aviation information and analytical results.

Comparative Summary

Feature	**Existing Aviation Data Services**	**Aero Flow**
Aviation Data Access	Provide specific aviation datasets or services.	Integrates multiple aviation data sources.
Historical Data	Availability depends on the provider.	Processes selected historical flight records.
Operational Event Streaming	Not consistently provided across the reviewed services.	Processes simulated operational events using Kafka and Spark.
Data Integration	Generally outside the scope of individual source services.	Integrates flight, airport, weather, and simulated operational data.
Distributed Storage	Depends on the provider and is not necessarily exposed to users.	Implements HDFS, Parquet, and Iceberg storage layers.

Historical Tracking	Not a shared feature across all reviewed sources.	Uses SCD Type 2 for selected analytical data.
Data Quality Validation	Varies between services.	Applies automated validation within the processing pipelines.
Interactive Dashboard	Varies by service.	Provides an integrated Streamlit dashboard.

The main contribution of Aero Flow is not the creation of a new aviation data provider, but the development of an end-to-end Data Engineering platform capable of integrating different aviation datasets and operational events into organized analytical outputs.

The reviewed solutions help guide the selection of appropriate data sources and identify important considerations related to source availability, schema consistency, and historical information.

2.3 References

1. **Open Sky Network.** *OpenSky Network API Documentation.* https://openskynetwork.github.io/opensky-api/
2. **U.S. Bureau of Transportation Statistics.** *Airline On-Time Performance Data.* https://www.transtats.bts.gov/ontime/
3. **OurAirports.** *Open Airport Data.* https://ourairports.com/data/
4. **Aviation Weather Center.** *Aviation Weather Data API.* https://aviationweather.gov/data/api/

5.3. Requirements Gathering

6. 3.1 Stakeholders and Their Needs

7. The Aero Flow platform is intended to support users interested in collecting, processing, managing, and analyzing aviation-related data.
8. The main stakeholders and their requirements are identified below.

Stakeholder	Main Needs
Data Engineers	Reliable ingestion, streaming, data integration, storage, and pipeline monitoring.
Analytics Engineers	Structured analytical datasets, transformation models, and historical records.

Data Analysts	Accurate and consistent aviation data for calculating performance indicators.
Aviation Data Researchers	Access to structured historical flight, airport, and weather datasets.
System Administrators	Ability to monitor operations, identify failures, and support recovery.
Project Team	A testable and maintainable platform with documented components and reproducible results.

9. The platform will address these needs within the scope of an educational Data Engineering prototype.

10. 3.2 Functional Requirements

11. Functional requirements describe the main operations and capabilities that Aero Flow is expected to provide.

ID	Functional Requirement
FR-01	Collect aviation reference data and historical flight records from selected publicly available datasets.
FR-02	Retrieve aviation weather observations through an external API.
FR-03	Validate, clean, and standardize collected data using Python batch ingestion pipelines.
FR-04	Generate simulated flight records and operational events using Python and FastAPI.
FR-05	Store and manage simulated flight information in PostgreSQL.
FR-06	Publish simulated operational flight events through Apache Kafka.
FR-07	Process incoming Kafka events using Spark Structured Streaming.
FR-08	Validate streaming records and handle duplicate and late-arriving events according to defined processing rules.
FR-09	Store raw, processed, and curated datasets using HDFS, Parquet, and Iceberg.
FR-10	Integrate flight, airport, weather, and simulated operational datasets using Apache Spark and Spark SQL.
FR-11	Build analytical data models using SQL and dbt.
FR-12	Maintain relevant historical flight changes using SCD Type 2.
FR-13	Execute automated data quality and consistency checks.

FR-14	Calculate flight performance indicators, including delays, cancellations, and operational trends.
FR-15	Display airport information, flight records, analytical indicators, and historical changes through Streamlit.
FR-16	Provide pipeline monitoring information and data quality results.
FR-17	Support controlled testing and recovery from selected processing failures.

12. The functional requirements will be validated during individual component testing and final system integration.

13. 3.3 Non-Functional Requirements

14. Non-functional requirements describe the quality attributes and operational characteristics expected from Aero Flow.

15. 1. Performance

16. The platform should process incoming flight events efficiently and provide measurable data processing latency and throughput.
17. Batch and streaming performance will be evaluated using controlled testing scenarios.

18. 2. Reliability

19. The system should support recovery from selected processing failures and provide error-handling mechanisms.
20. Checkpointing and recovery testing will be used to evaluate processing reliability.

21. 3. Scalability

22. The architecture should support increasing data volumes and additional data sources without requiring major structural changes.
23. Scalability will be evaluated within the available computing resources.

24. 4. Data Quality

25. The platform should maintain consistent and validated data throughout the ingestion, processing, storage, and analytical stages.
26. Automated validation should identify missing values, duplicates, invalid records, and inconsistent relationships.

27. 5. Security

28. The system should protect database credentials and sensitive configuration information.
29. Access to system components and stored data should be controlled according to the project environment.

30. 6. Usability

31. The dashboard should provide clear navigation, understandable visualizations, and accessible information about aviation data and processing results.
32. Users should be able to interpret important indicators without requiring detailed technical knowledge of the underlying pipelines.

33. 7. Maintainability

34. The project should use modular code, documented components, consistent data schemas, and GitHub version control.

35. Each component should include sufficient instructions for execution, testing, and troubleshooting.

36. 8. Data Consistency

37. The platform should apply consistent validation and transformation rules across integrated datasets.
38. Simulated flight events should remain distinguishable from real historical aviation data.

39. 9. Recoverability

40. The system should support restarting selected processing components after controlled failures.
41. Recovery testing should verify data consistency and identify missing or duplicate processing results.

42. 10. Testability

43. The main project components should be independently testable before full integration.
44. The team should maintain suitable test datasets, execution instructions, and validation results.

45. Requirements Validation

46. Functional and non-functional requirements will be evaluated through component testing and end-to-end integration testing.
47. Performance and reliability measurements will be based on actual execution results.
48. Specific numerical targets will be confirmed according to the available computing resources and agreed evaluation criteria.
49. The final prototype will prioritize functional correctness, reliable data processing, analytical consistency, and successful integration of the main Data Engineering components.